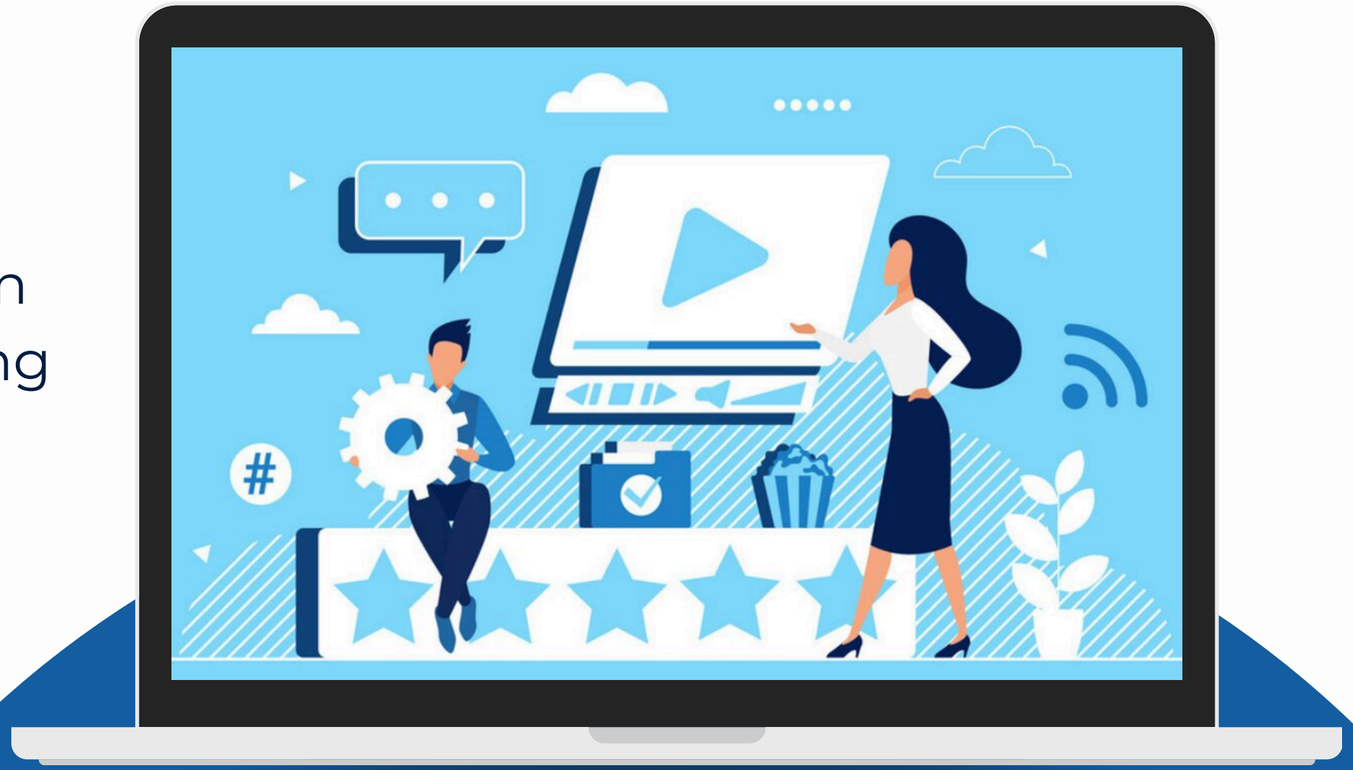
CSE-342 INTELLIGENT COMPUTING
MINI PROJECT

Fairness Aware Movie Recommendation System using neural collaborative filtering and decomposed adversarial learning

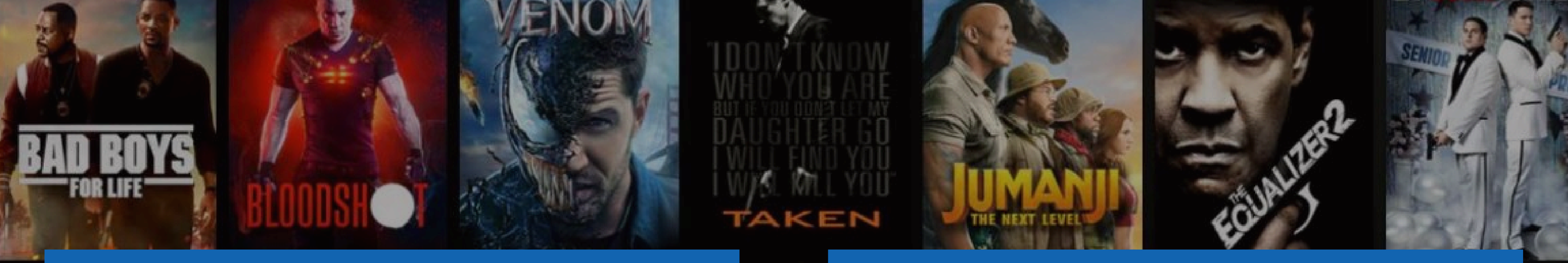
By: Lakshya Singhal (23075043)
Sairaj Chavan(23075070)
Shanawaz (23075076)



Project Overview

- Introduction+Objective 1
- Fundamental Framework 2
- Components 3-4
- Plot of different metrics 5
- Neural Collaborative Filtering 5-7
- Model flowchart 8
- Fairness Evaluation 9
- Fairness Metrics 10
- Results 11





Introduction

- User behavior patterns among individuals with the same sensitive attributes, like gender, can lead to biases in movie recommendations (e.g., recommending action movies predominantly to male users).
- This unfairness limits users' exposure to diverse genres. To address this, we propose a fairness-aware movie recommendation approach using decomposed adversarial learning and orthogonality regularization to reduce biases from sensitive user attributes.

Objective

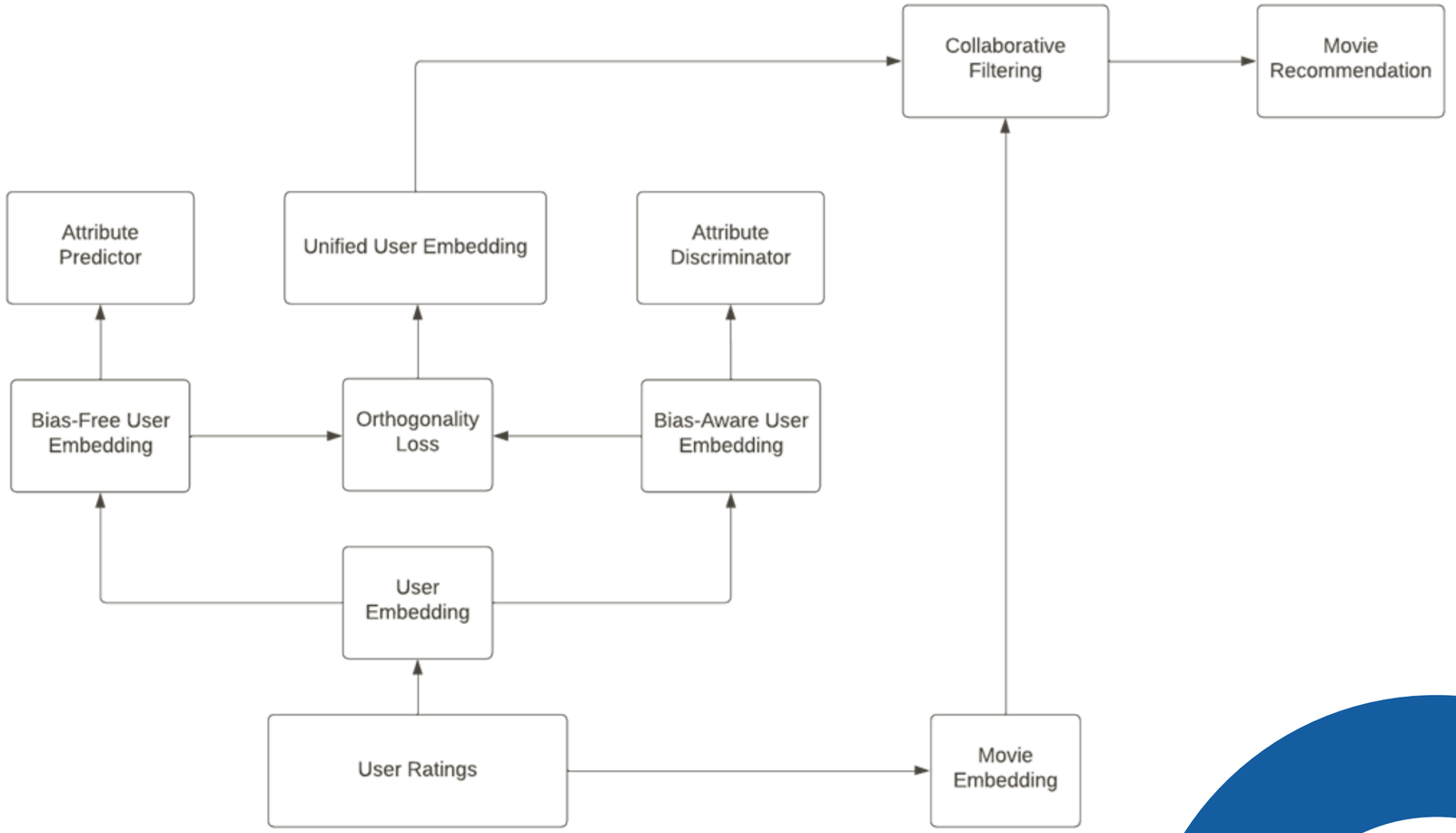
- Major movie streaming companies may hesitate to invest in new methods for generating user embeddings due to perceived lack of profitability and higher costs.
- Current recommendation systems rely on user click patterns and sensitive attributes, resulting in limited diversity and potential subscriber loss.
- Emphasizing fairness can enhance both fairness and diversity in movie recommendations, helping to retain subscribers.

Fundamental Framework

We propose a user interest model with two components:

- a bias-aware user embedding that captures bias from sensitive attributes(enhancing bias modeling)
- a bias-free embedding encoding attribute-independent interests for fair movie recommendations (removing bias from the bias-free embedding)

Orthogonality regularization ensures the two embeddings remain distinct.



Components



Model

- Fix embedding size for users and movies (p).
- Create a px1 vector for each user and movie.
- Attribute predictor: a simple neural network with no hidden layers, using p inputs (bias-aware user embedding) and a bias term, activated by the sigmoid function for outputs ranging from 0-1.
- Attribute discriminator: another simple neural network with p inputs (bias-free user embedding) and a bias term, also activated by the sigmoid function for outputs between 0-1.

Collaborative Filtering

- Collaborative filtering utilizes similarities between users and movies for accurate recommendations, leveraging the interests of similar users.
- The model learns embeddings automatically without manual feature engineering.

Nu and Nm represent the number of users and movies; ui and mj are user and movie embeddings, while rij denotes the user i's rating for movie j, with only rated user-movie pairs included in the loss function.

Attribute Predictor

- Attribute Predictor and Discriminator are neural networks that learn sensitive attributes from user embeddings.
- The attribute predictor minimizes the loss between inferred and given attributes.
- The formula is z = WbUb + Bb, where Ub is the bias-aware user embedding, and Wb and Bb are neural network parameters using a sigmoid activation function.

Nu denotes the number of users, with z representing gold-label values and z the predicted sensitive attribute values.

Components



Attribute Discriminator

- Nu indicates the number of users; z represents gold-label values and predicted values of the sensitive attribute.
- For the attribute discriminator, the loss between inferred and actual attributes is maximized.

Formula: z = WdUd + Bd, where Ud is the bias-free user embedding, and Wd and Bd are neural network parameters with a sigmoid activation function.

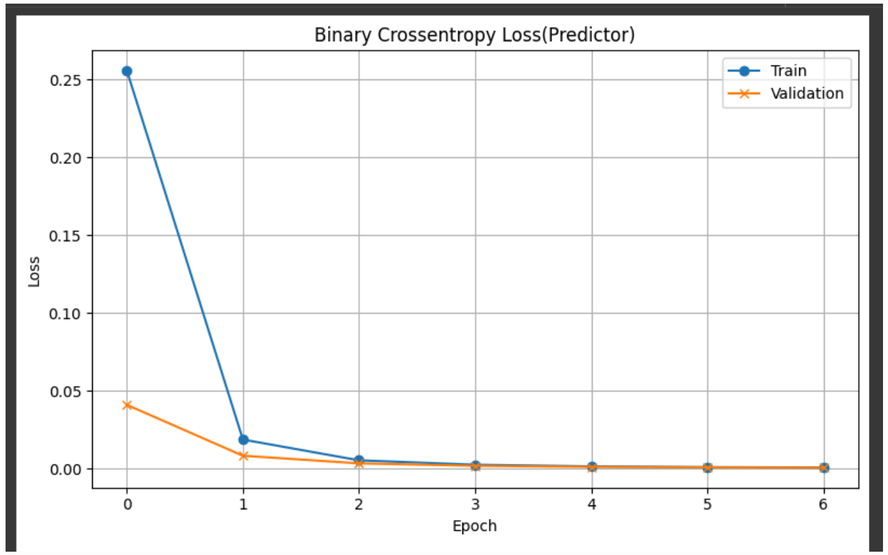
Loss Functions

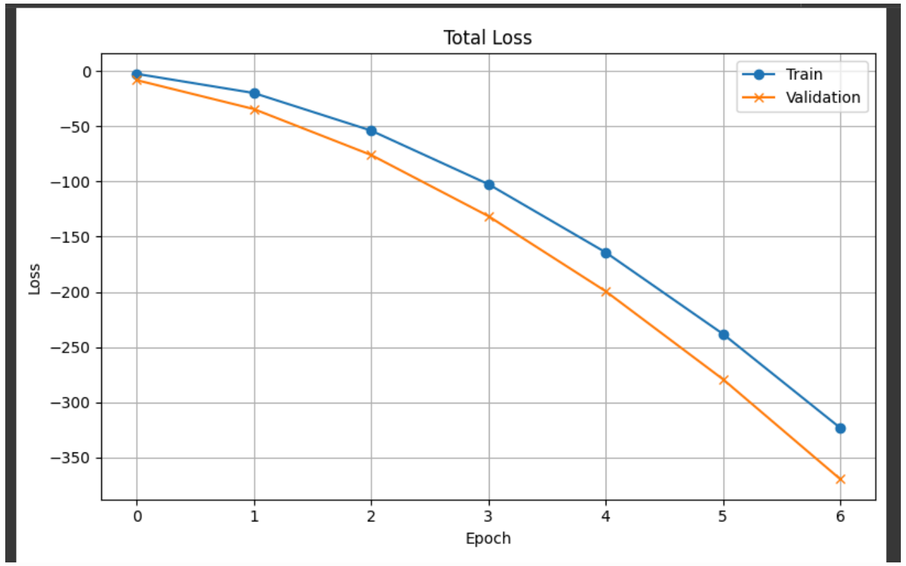
- The loss function consists of four parts:
  - Collaborative filtering loss function
  - Attribute prediction loss function
  - Attribute discrimination loss function (similar to attribute prediction but with a negative sign for sensitive information loss)
  - Loss function to make bias-aware and bias-free user embeddings orthogonal.

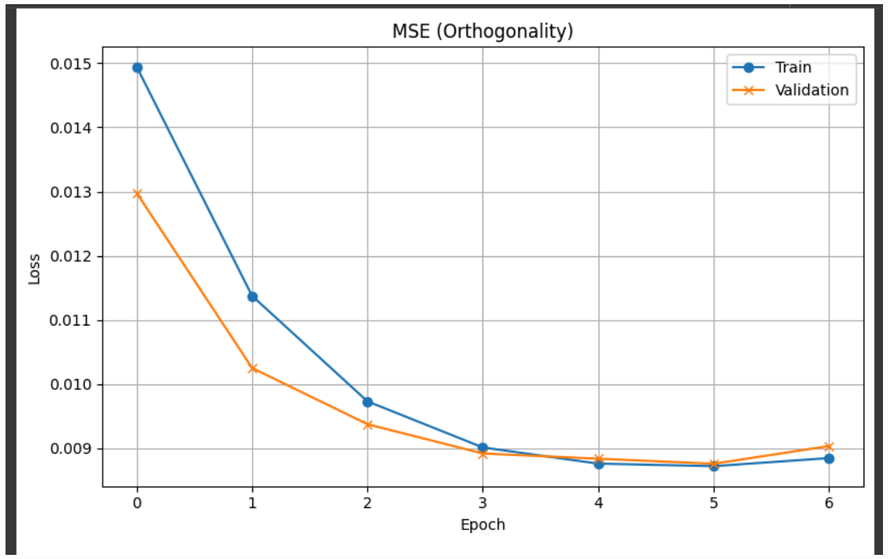
Orthogonality Regularisation

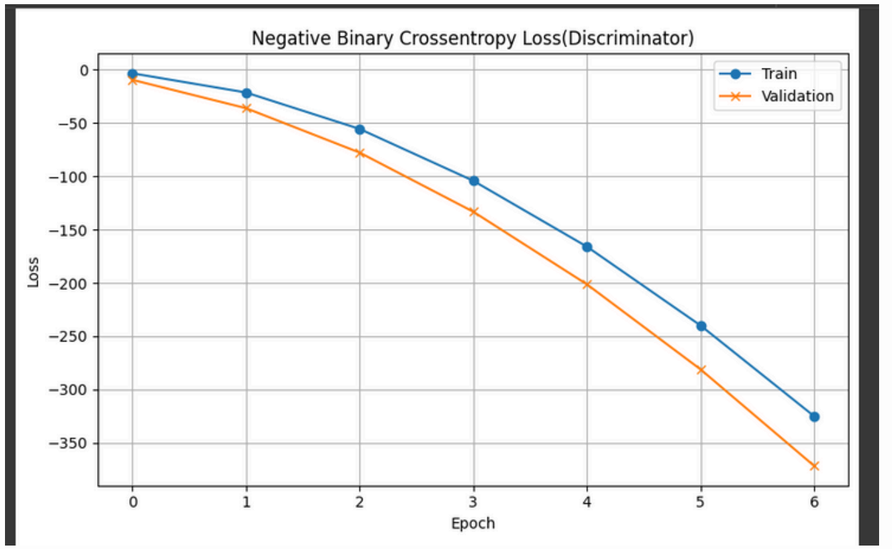
- It regularises the bias-aware user embedding and bias-free user embedding by encouraging them to be orthogonal to each other.

Plot of different metrics





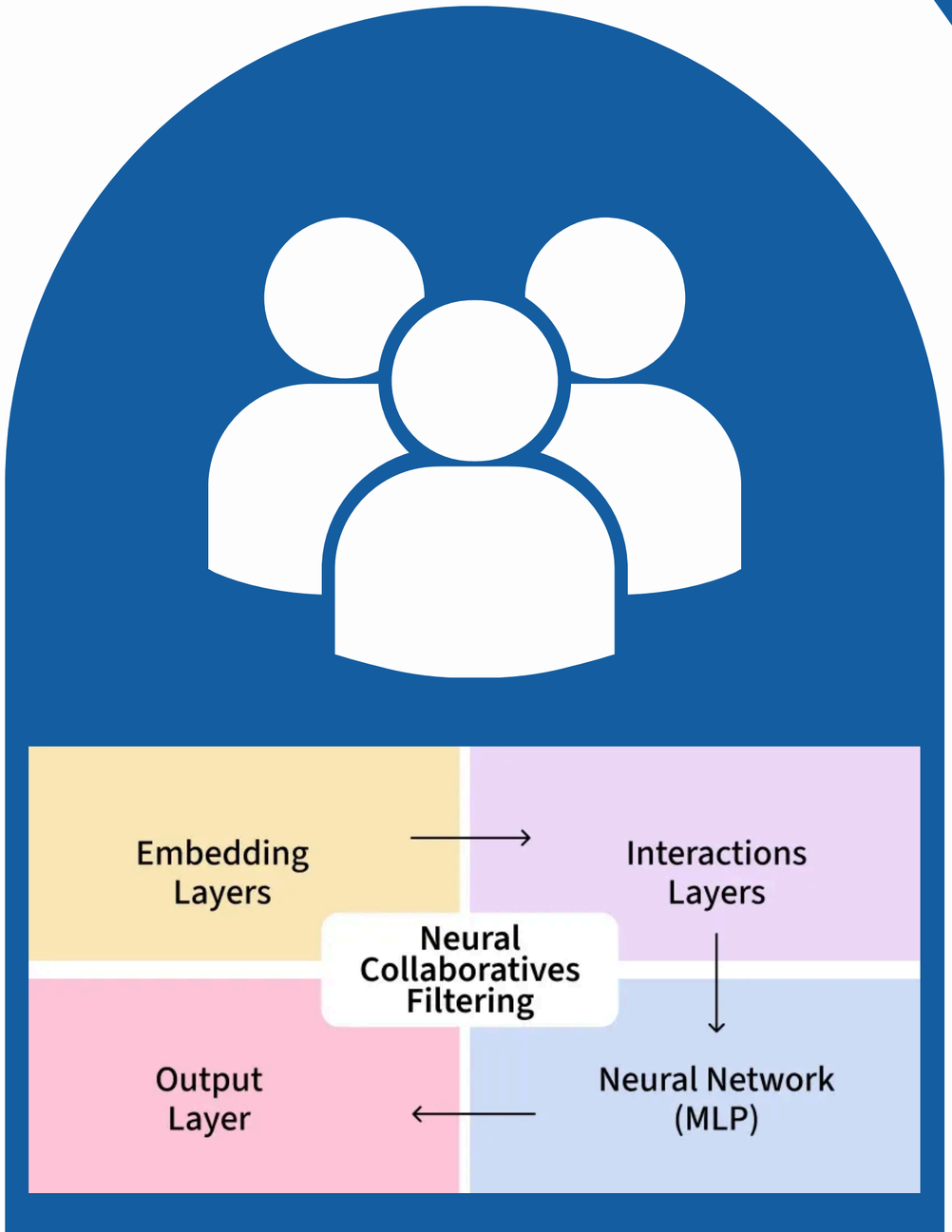


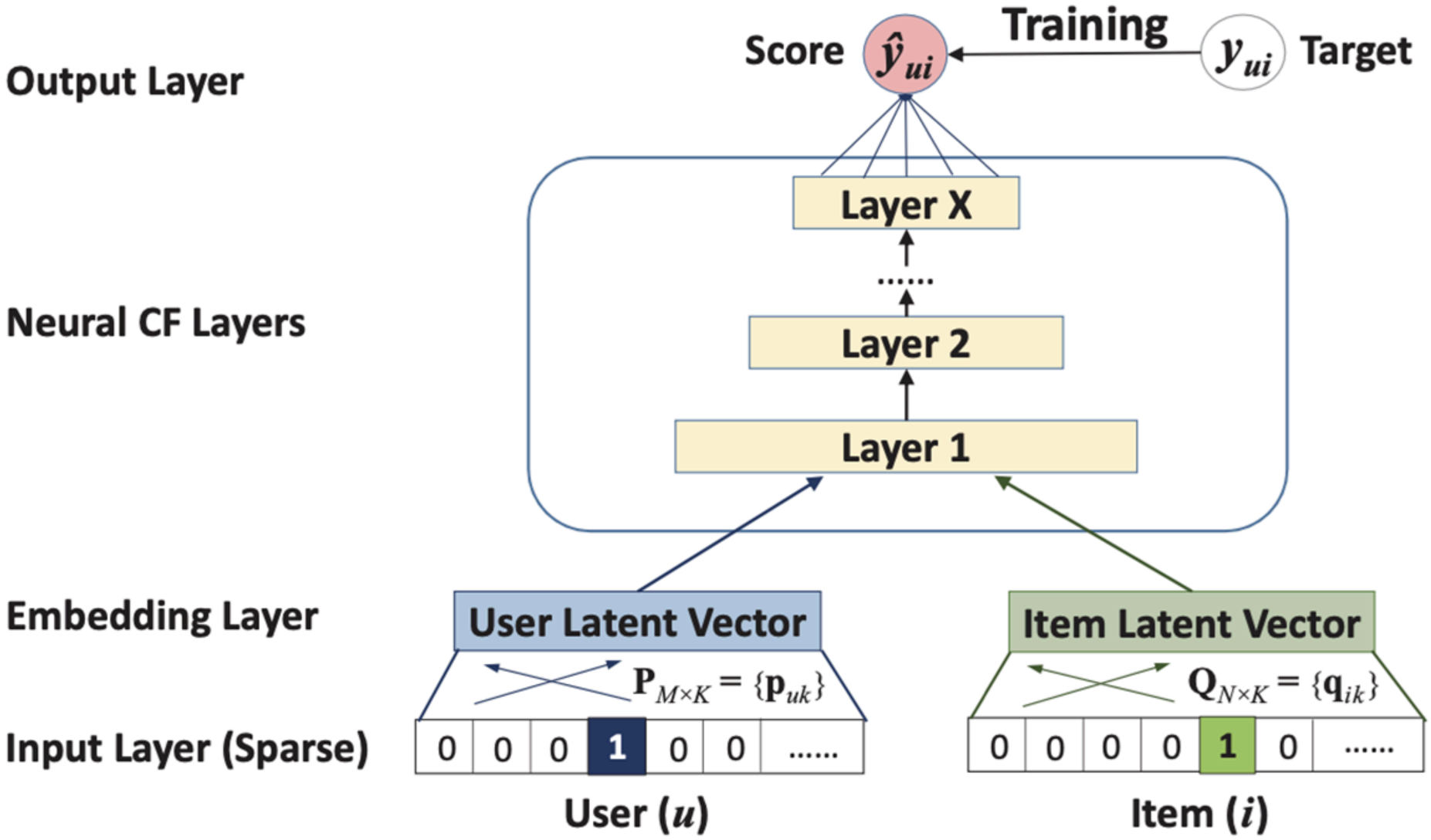


WHY Neural Collaborating Filtering?

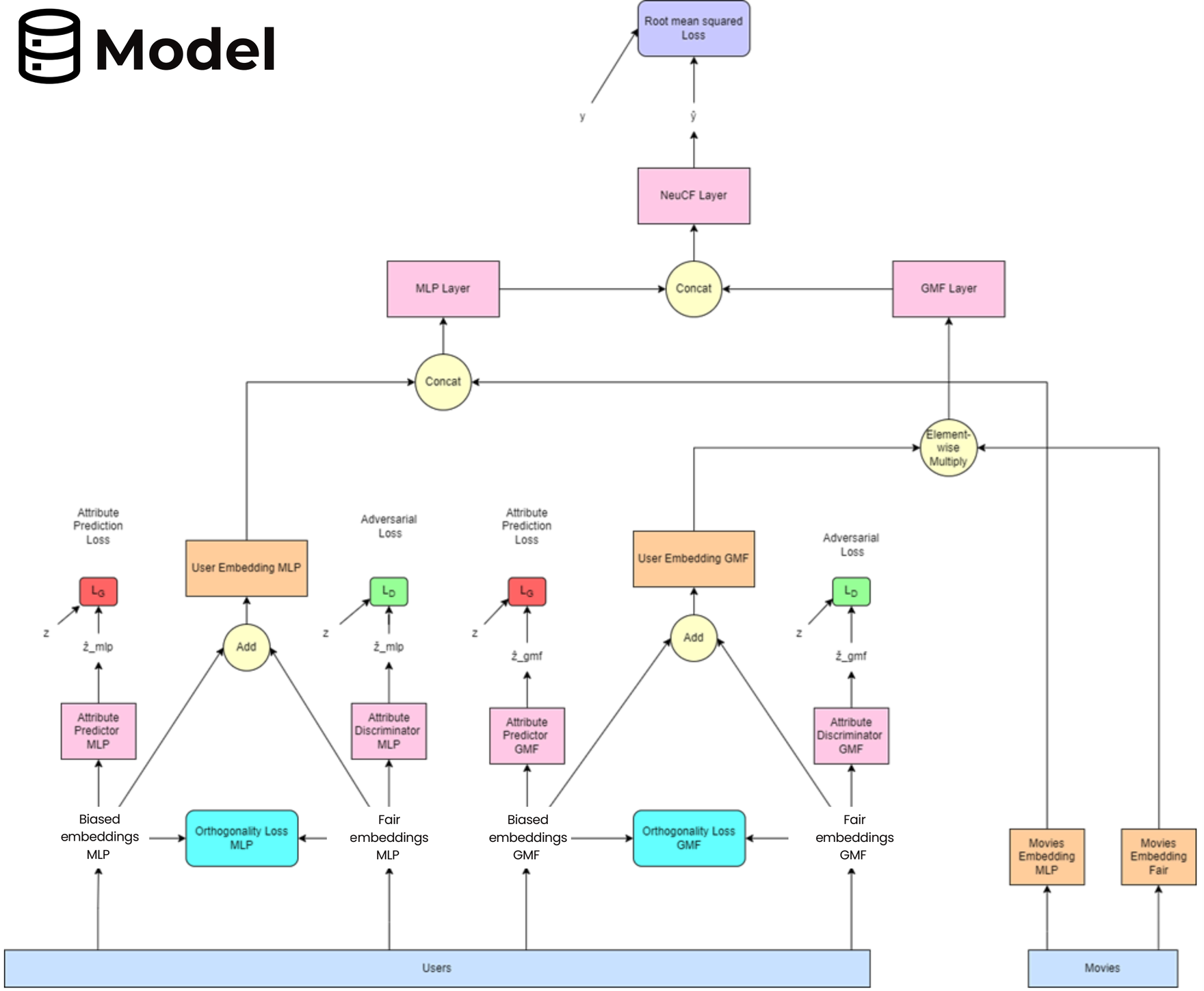
- Matrix factorization is a well-known collaborative filtering technique.
- The inner product used in matrix factorization can limit performance.
- This linear combination of latent features may fail to capture complex user interactions.

Neural collaborative filtering utilizes deep neural networks to learn the interaction function from data.




Output Layer
Score
$\hat{y}_{ui}$
Training
y_{ui}
Target
Neural CF Layers
Layer X
.....
Layer 2
Layer 1
Embedding Layer
User Latent Vector
$\mathbf{P}_{M\times K} = \{\mathbf{p}_{uk}\}$
Item Latent Vector
$\mathbf{Q}_{N\times K} = \{\mathbf{q}_{ik}\}$
Input Layer (Sparse)
0 0 0 1 0 0 .....
0 0 0 0 1 0 .....
User (u)
Item (i)

Model


Root mean squared Loss
y
ŷ
NeuCF Layer
MLP Layer
Concat
GMF Layer
Concat
Element-wise Multiply
Attribute Prediction Loss
User Embedding MLP
Adversarial Loss
Attribute Prediction Loss
User Embedding GMF
Adversarial Loss
z
ż_mlp
Add
ż_mlp
z
ż_gmf
Add
ż_gmf
z
Attribute Predictor MLP
Attribute Discriminator MLP
Attribute Predictor GMF
Attribute Discriminator GMF
Biased embeddings MLP
Orthogonality Loss MLP
Fair embeddings MLP
Biased embeddings GMF
Orthogonality Loss GMF
Fair embeddings GMF
Movies Embedding MLP
Movies Embedding Fair
Users
Movies

Fairness Evaluation

```
gender_predictions = discriminator.predict(fair_embeddings)
score = 0
for x in gender_predictions:
    score += abs(x-0.5)

score /= len(gender_predictions)
print(score)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
[0.08156089]
```

Fairness Metrics

01

Unfairness

$$U_{\text{val}} = \frac{1}{n}\sum_{j=1}^{n}\left|\left(\mathrm{E}_g\left[y\right]_j - \mathrm{E}_g\left[r\right]_j\right) - \left(\mathrm{E}_{\neg g}\left[y\right]_j - \mathrm{E}_{\neg g}\left[r\right]_j\right)\right|$$

02

Absolute unfairness

$$U_{\text{abs}} = \frac{1}{n}\sum_{j=1}^{n}\left|\left|\mathrm{E}_g\left[y\right]_j - \mathrm{E}_g\left[r\right]_j\right| - \left|\mathrm{E}_{\neg g}\left[y\right]_j - \mathrm{E}_{\neg g}\left[r\right]_j\right|\right|$$

03

Underestimation fairness

$$U_{\text{under}} = \frac{1}{n}\sum_{j=1}^{n}\left|\max\{0, \mathrm{E}_g\left[r\right]_j - \mathrm{E}_g\left[y\right]_j\} - \max\{0, \mathrm{E}_{\neg g}\left[r\right]_j - \mathrm{E}_{\neg g}\left[y\right]_j\}\right|$$

04

Overestimation Unfairness

$$U_{\text{over}} = \frac{1}{n}\sum_{j=1}^{n}\left|\max\{0, \mathrm{E}_g\left[y\right]_j - \mathrm{E}_g\left[r\right]_j\} - \max\{0, \mathrm{E}_{\neg g}\left[y\right]_j - \mathrm{E}_{\neg g}\left[r\right]_j\}\right|$$

Dataset

- Movie Lens Dataset:
- Dataset
- The file ‘u.data’ contains the ratings users gave to each movie along with the time stamp
- The file ‘u.user’ contains information regarding user’s attributes including their gender

Results

```
    under_est += np.abs(np.maximum(0, eg_r - eg_y)

val_score /= l
abs_score /= l
over_est /= l
under_est /= l

print(f'Unfairness: {val_score}')
print(f'Absolute Unfairness: {abs_score}')
print(f'Overestimation: {over_est}')
print(f'Underestimation: {under_est}')
```

```
Unfairness: 0.20774929326906377
Absolute Unfairness: 0.1841541786502013
Overestimation: 0.14388151887503137
Underestimation: 0.06386777439403253
```

Thank You!